

Mảng hậu tố: công cụ mạnh để xử lý xâu

Luo Suiqian

Trường Trung học Phụ thuộc Đại học Sư phạm Hoa Nam

Luận văn đội tuyển dự tuyển quốc gia IOI 2009

Nguồn gốc: Mảng hậu tố: công cụ mạnh để xử lý xâu

IOI China candidate team papers / 2009 / Luo Suiqian

Abstract

Mảng hậu tố là một công cụ rất mạnh trong xử lý xâu. Nó là một thay thế tinh tế cho cây hậu tố: dễ cài đặt hơn, có thể thực hiện nhiều chức năng tương tự với độ phức tạp thời gian không kém nhiều, và dùng ít bộ nhớ hơn đáng kể. Do đó trong thi Tin học, mảng hậu tố thường thực dụng hơn cây hậu tố.

Bài viết gồm hai phần. Phần thứ nhất giới thiệu hai cách xây dựng mảng hậu tố: thuật toán nhân đôi và thuật toán DC3, nhấn mạnh cách viết mã ngắn gọn, hiệu quả, rồi so sánh hai thuật toán. Phần thứ hai trình bày các ứng dụng thường gặp của mảng hậu tố trong những lớp bài toán xử lý xâu.

Từ khóa. Xâu; hậu tố; mảng hậu tố; mảng hạng; sắp xếp cơ số; DC3; tiền tố chung dài nhất.

Contents

1 Cài đặt mảng hậu tố	1
1.1 Các định nghĩa cơ bản	1
1.2 Thuật toán nhân đôi	2
1.3 Thuật toán DC3	4
1.4 So sánh thuật toán nhân đôi và DC3	6
2 Ứng dụng của mảng hậu tố	7
2.1 Tiền tố chung dài nhất	7
2.2 Các bài toán trên một xâu	8
2.2.1 Xâu con lặp	8
2.2.2 Số lượng xâu con	8
2.2.3 Xâu con đối xứng	8
2.2.4 Xâu con lặp liên tiếp	9
2.3 Các bài toán trên hai xâu	9
2.3.1 Xâu con chung	9
2.3.2 Số lượng xâu con chung	10
2.4 Các bài toán trên nhiều xâu	10

1 Cài đặt mảng hậu tố

Phần này trình bày hai phương pháp xây dựng mảng hậu tố: thuật toán nhân đôi và thuật toán DC3. Cả hai đôi khi bị xem là khó hiểu và khó cài đặt. Vì vậy ngoài ý tưởng, phần này giữ lại các đoạn mã C ngắn gọn trong bài gốc: bản nhân đôi khoảng 25 dòng, còn bản DC3 khoảng 40 dòng.

1.1 Các định nghĩa cơ bản

Xâu con. Với chuỗi r , chuỗi con $r[i..j]$, $i \leq j$, là đoạn từ vị trí i tới vị trí j , tức chuỗi tạo bởi

$$r[i], r[i+1], \dots, r[j].$$

Hậu tố. Hậu tố là chuỗi con đặc biệt bắt đầu tại một vị trí i và kéo dài tới cuối chuỗi. Hậu tố của r bắt đầu ở ký tự thứ i được viết

$$\text{suffix}(i) = r[i.. \text{len}(r)].$$

So sánh từ điển. So sánh hai chuỗi u, v theo thứ tự từ điển nghĩa là lần lượt so sánh $u[i]$ và $v[i]$ từ $i = 1$. Nếu hai ký tự bằng nhau thì tăng i , nếu $u[i] < v[i]$ thì $u < v$, còn nếu $u[i] > v[i]$ thì $u > v$. Nếu một trong hai chuỗi đã hết mà vẫn chưa phân định được, chuỗi ngắn hơn được coi là nhỏ hơn; nếu hai độ dài bằng nhau thì hai chuỗi bằng nhau.

Từ định nghĩa này, hai hậu tố của cùng một chuỗi nhưng bắt đầu ở hai vị trí khác nhau không thể bằng nhau, vì điều kiện cần để hai chuỗi bằng nhau là chúng có cùng độ dài.

Mảng hậu tố. Cho chuỗi S dài n . Mảng hậu tố SA là một hoán vị của $1, 2, \dots, n$ thỏa mãn

$$\text{suffix}(\text{SA}[i]) < \text{suffix}(\text{SA}[i+1]), \quad 1 \leq i < n.$$

Nói cách khác, ta sắp xếp mọi hậu tố theo thứ tự tăng dần rồi ghi lại vị trí bắt đầu của chúng.

Mảng hạng. $\text{Rank}[i]$ là hạng của $\text{suffix}(i)$ trong thứ tự tăng dần của mọi hậu tố. Một cách nhớ ngắn gọn là: mảng hậu tố trả lời “ở hạng này là ai?”, còn mảng hạng trả lời “hậu tố này đứng thứ mấy?”. Hai mảng là nghịch đảo của nhau. Nếu đã có một trong hai mảng, ta tính mảng còn lại trong $\mathcal{O}(n)$.

Khi cài đặt, để so sánh thuận tiện, có thể thêm vào cuối chuỗi một ký tự đặc biệt chưa xuất hiện trước đó và nhỏ hơn mọi ký tự khác. Sau khi có mảng hạng, ta so sánh hai hậu tố bất kỳ trong $\mathcal{O}(1)$ bằng cách so sánh hạng. Nếu so sánh trực tiếp từng ký tự thì trong trường hợp xấu nhất cần tới n phép so sánh, nhưng chắc chắn không cần hơn.

1.2 Thuật toán nhân đôi

Ý tưởng chính của thuật toán nhân đôi là sắp xếp các đoạn con độ dài 2^k bắt đầu ở mọi vị trí và tính hạng của chúng. Ban đầu $k = 0$, tức mỗi đoạn chỉ là một ký tự. Mỗi lần tăng k lên 1. Khi $2^k > n$, mỗi đoạn độ dài 2^k tương đương với một hậu tố hoàn chỉnh; nếu mọi hạng đã khác nhau thì ta đã có kết quả cuối cùng.

Trong mỗi vòng, hạng của đoạn độ dài 2^k được biểu diễn bằng hai khóa: hạng của nửa đầu độ dài 2^{k-1} và hạng của nửa sau độ dài 2^{k-1} . Sắp xếp hai khóa này bằng sắp xếp cơ sở sẽ cho thứ tự mới. Với chuỗi aabaaaab, các khóa x, y ở từng vòng lần lượt mô tả hai nửa của đoạn đang xét; hình trong bản gốc minh họa rằng các hạng dần tách ra cho tới khi mọi hậu tố có hạng riêng.

Mã cài đặt.

```
int wa[maxn],wb[maxn],wv[maxn],ws[maxn];
int cmp(int *r,int a,int b,int l)
{return r[a]==r[b]&&r[a+l]==r[b+l];}
void da(int *r,int *sa,int n,int m)
{
    int i,j,p,*x=wa,*y=wb,*t;
    for(i=0;i<m;i++) ws[i]=0;
    for(i=0;i<n;i++) ws[x[i]=r[i]]++;
    for(i=1;i<m;i++) ws[i]+=ws[i-1];
    for(i=n-1;i>=0;i--) sa[--ws[x[i]]]=i;
    for(j=1,p=1;p<n;j*=2,m=p)
    {
        for(p=0,i=n-j;i<n;i++) y[p++]=i;
        for(i=0;i<n;i++) if(sa[i]>=j) y[p++]=sa[i]-j;
        for(i=0;i<n;i++) wv[i]=x[y[i]];
        for(i=0;i<m;i++) ws[i]=0;
        for(i=0;i<n;i++) ws[wv[i]]++;
        for(i=1;i<m;i++) ws[i]+=ws[i-1];
        for(i=n-1;i>=0;i--) sa[--ws[wv[i]]]=y[i];
        for(t=x,x=y,y=t,p=1,x[sa[0]]=0,i=1;i<n;i++)
            x[sa[i]]=cmp(y,sa[i-1],sa[i],j)?p-1:p++;
    }
    return;
}
```

Xâu cần sắp được đặt trong mảng $r[0..n-1]$, mọi giá trị đều nhỏ hơn m . Để hàm thao tác thuận tiện, quy ước $r[n-1] = 0$, còn các $r[i]$ khác đều dương. Kết quả nằm trong $sa[0..n-1]$.

Bước đầu tiên sắp xếp các xâu độ dài 1. Trong bài toán xâu thông thường, giá trị ký tự không lớn, nên dùng sắp xếp cơ số:

```
for(i=0;i<m;i++) ws[i]=0;
for(i=0;i<n;i++) ws[x[i]=r[i]]++;
for(i=1;i<m;i++) ws[i]+=ws[i-1];
for(i=n-1;i>=0;i--) sa[--ws[x[i]]]=i;
```

Ở đây x đóng vai trò mảng hạng tạm thời. Ta chưa cần nén thành hạng thật sự, vì những bước sau chỉ dùng x để so sánh độ lớn của ký tự.

Trong mỗi vòng nhân đôi, sắp xếp cơ số trên hai khóa thường gồm hai lượt: sắp theo khóa thứ hai rồi theo khóa thứ nhất. Kết quả sắp theo khóa thứ hai có thể suy ra trực tiếp từ SA của vòng trước, không cần sắp lại:

```
for(p=0,i=n-j;i<n;i++) y[p++]=i;
for(i=0;i<n;i++) if(sa[i]>=j) y[p++]=sa[i]-j;
```

Biến j là độ dài nửa đoạn hiện tại, còn y lưu thứ tự đã sắp theo khóa thứ hai. Sau đó chỉ cần sắp theo khóa thứ nhất:

```
for(i=0;i<n;i++) wv[i]=x[y[i]];
```

```

for(i=0;i<m;i++) ws[i]=0;
for(i=0;i<n;i++) ws[wv[i]]++;
for(i=1;i<m;i++) ws[i]+=ws[i-1];
for(i=n-1;i>=0;i--) sa[-ws[wv[i]]]=y[i];

```

Khi đã có SA mới, ta tính hạng mới. Có thể có nhiều đoạn nhận cùng hạng, nên cần kiểm tra hai đoạn có giống hệt nhau hay không. Để tiết kiệm bộ nhớ, mảng y được dùng lại làm mảng hạng cũ; hai con trỏ x, y được hoán đổi thay vì sao chép cả mảng:

```

for(t=x, x=y, y=t, p=1, x[sa[0]]=0, i=1; i<n; i++)
x[sa[i]]=cmp(y, sa[i-1], sa[i], j)?p-1:p++;

```

Hàm so sánh là

```

int cmp(int *r, int a, int b, int l)
{return r[a]==r[b]&& r[a+l]==r[b+l];}

```

Quy ước $r[n-1] = 0$ giúp tránh kiểm tra biên: nếu $r[a] = r[b]$ thì đoạn độ dài l bắt đầu ở a và b chưa chứa ký tự kết thúc, nên truy cập $r[a+l]$ và $r[b+l]$ vẫn hợp lệ. Sau vòng này, p là số hạng khác nhau. Nếu $p = n$, mọi đoạn đã phân biệt, các vòng sau không làm thay đổi thứ tự nữa. Vì vậy điều kiện vòng lặp được viết:

```

for(j=1, p=1; p<n; j*=2, m=p) { ... }

```

Mỗi vòng dùng sắp xếp cơ sở trong $\mathcal{O}(n)$; số vòng nhiều nhất là $\mathcal{O}(\log n)$. Độ phức tạp thời gian của thuật toán nhân đôi là $\mathcal{O}(n \log n)$, bộ nhớ là tuyến tính.

1.3 Thuật toán DC3

DC3 là thuật toán xây dựng mảng hậu tố tuyến tính theo tư tưởng chia để trị. Thuật toán gồm ba bước.

Bước 1: sắp xếp các hậu tố có vị trí không chia hết cho 3. Chia hậu tố thành hai phần. Phần thứ nhất gồm các hậu tố bắt đầu ở vị trí k với $k \bmod 3 \neq 0$, tức $\text{suffix}(1), \text{suffix}(2), \text{suffix}(4), \text{suffix}(5), \text{suffix}(7), \dots$. Ta ghép hai dãy vị trí $1 \bmod 3$ và $2 \bmod 3$. Nếu độ dài không phải bội của 3 thì thêm các số 0 ở cuối để đủ nhóm. Sau đó cứ ba ký tự được coi là một bộ ba, sắp xếp cơ sở các bộ ba này và nén mỗi bộ ba thành một ký tự mới. Mảng hậu tố của xâu mới được tính đệ quy. Từ đó suy ra thứ tự của mọi hậu tố có vị trí bắt đầu không chia hết cho 3. Xâu gốc phải kết thúc bằng một ký tự nhỏ nhất và chưa từng xuất hiện để bảo đảm thứ tự suy ra là đúng.

Bước 2: sắp xếp các hậu tố có vị trí chia hết cho 3. Mỗi hậu tố còn lại có dạng một ký tự đi kèm một hậu tố đã có hạng ở bước 1. Do đó chỉ cần một lượt sắp xếp cơ sở để có thứ tự của nhóm này.

Bước 3: trộn hai kết quả. Thao tác trộn giống bước trộn trong mergesort. Cần so sánh nhanh một hậu tố thuộc nhóm $0 \bmod 3$ với một hậu tố thuộc nhóm còn lại. Có hai trường hợp:

$$\begin{aligned}\text{suffix}(3i) &= r[3i] + \text{suffix}(3i+1), \\ \text{suffix}(3j+1) &= r[3j+1] + \text{suffix}(3j+2),\end{aligned}$$

trong đó phần sau đã có hạng từ bước 1; và

$$\begin{aligned}\text{suffix}(3i) &= r[3i] + r[3i+1] + \text{suffix}(3i+2), \\ \text{suffix}(3j+2) &= r[3j+2] + r[3j+3] + \text{suffix}(3(j+1)+1).\end{aligned}$$

Lý do phải gộp theo bộ ba thay vì bộ đôi nằm ở đây: sau khi nhìn một hoặc hai ký tự đầu, phần còn lại rơi vào nhóm đã được xếp hạng ở bước 1.

Mã cài đặt.

```
#define F(x) ((x)/3+((x)%3==1?0:tb))
#define G(x) ((x)<tb?(x)*3+1:((x)-tb)*3+2)
int wa[maxn],wb[maxn],wv[maxn],ws[maxn];
int c0(int *r,int a,int b)
{return r[a]==r[b]&&r[a+1]==r[b+1]&&r[a+2]==r[b+2];}
int c12(int k,int *r,int a,int b)
{if(k==2) return r[a]<r[b]||r[a]==r[b]&&c12(1,r,a+1,b+1);
else return r[a]<r[b]||r[a]==r[b]&&wv[a+1]<wv[b+1];}
void sort(int *r,int *a,int *b,int n,int m)
{
    int i;
    for(i=0;i<n;i++) wv[i]=r[a[i]];
    for(i=0;i<m;i++) ws[i]=0;
    for(i=0;i<n;i++) ws[wv[i]]++;
    for(i=1;i<m;i++) ws[i]+=ws[i-1];
    for(i=n-1;i>=0;i--) b[--ws[wv[i]]]=a[i];
    return;
}
void dc3(int *r,int *sa,int n,int m)
{
    int i,j,*rn=r+n,*san=sa+n,ta=0,tb=(n+1)/3,tbc=0,p;
    r[n]=r[n+1]=0;
    for(i=0;i<n;i++) if(i%3!=0) wa[tbc++]=i;
    sort(r+2,wa,wb,tbc,m);
    sort(r+1,wb,wa,tbc,m);
    sort(r,wa,wb,tbc,m);
    for(p=1,rn[F(wb[0])]=0,i=1;i<tbc;i++)
        rn[F(wb[i])]=c0(r,wb[i-1],wb[i])?p-1:p++;
    if(p<tbc) dc3(rn,san,tbc,p);
    else for(i=0;i<tbc;i++) san[rn[i]]=i;
    for(i=0;i<tbc;i++) if(san[i]<tb) wb[ta++]=san[i]*3;
    if(n%3==1) wb[ta++]=n-1;
    sort(r,wb,wa,ta,m);
    for(i=0;i<tbc;i++) wv[wb[i]=G(san[i])]=i;
    for(i=0,j=0,p=0;i<ta && j<tbc;p++)
        sa[p]=c12(wb[j]%3,r,wa[i],wb[j])?wa[i++]:wb[j++];
    for(;i<ta;p++) sa[p]=wa[i++];
    for(;j<tbc;p++) sa[p]=wb[j++];
    return;
}
```

Các tham số có ý nghĩa giống hàm nhân đôi. Khác biệt là r và sa nên có kích thước khoảng $3n$, để vùng nhớ phía sau được dùng cho xâu con đệ quy mà không phải cấp phát lại. Trong hàm, $rn = r + n$ lưu

xâu mới ở bước 1, còn $san = sa + n$ lưu mảng hậu tố của xâu mới. Biến ta là số hậu tố ở vị trí $0 \bmod 3$, $tb = (n + 1)/3$ là số hậu tố ở vị trí $1 \bmod 3$, và tbc là tổng số hậu tố thuộc hai lớp $1 \bmod 3$ và $2 \bmod 3$.

Để tạo xâu mới, trước hết đặt hai ký tự canh ở cuối bằng 0, liệt kê các vị trí không chia hết cho 3, rồi sắp xếp cơ sở ba lần theo ba ký tự:

```
r[n]=r[n+1]=0;
for(i=0;i<n;i++) if(i%3!=0) wa[tbc++]=i;
sort(r+2,wa,wb,tbc,m);
sort(r+1,wb,wa,tbc,m);
sort(r,wa,wb,tbc,m);
```

Hàm `sort` chỉ là một lượt sắp xếp cơ sở theo khóa $r[a[i]]$.

Sau khi các bộ ba đã được sắp, ta nén chúng thành các hạng mới:

```
for(p=1,rn[F(wb[0])]=0,i=1;i<tbc;i++)
rn[F(wb[i])]=c0(r,wb[i-1],wb[i])?p-1:p++;
```

Ở đây $F(x)$ đổi vị trí của hậu tố trong xâu gốc sang vị trí tương ứng trong xâu mới, còn $c0$ kiểm tra hai bộ ba có giống hệt nhau hay không. Nếu mọi bộ ba đều khác nhau, ta đảo trực tiếp mảng hạng thành san ; nếu không, gọi đệ quy:

```
if(p<tbc) dc3(rn,san,tbc,p);
else for(i=0;i<tbc;i++) san[rn[i]]=i;
```

Tiếp theo là sắp nhóm vị trí chia hết cho 3. Thứ tự theo khóa thứ hai có thể suy ra từ san , rồi chỉ cần sắp theo ký tự đầu. Trường hợp $n \bmod 3 = 1$ cần thêm hậu tố cuối $n - 1$, vì không có hậu tố n trong san :

```
for(i=0;i<tbc;i++) if(san[i]<tb) wb[ta++]=san[i]*3;
if(n%3==1) wb[ta++]=n-1;
sort(r,wb,wa,ta,m);
for(i=0;i<tbc;i++) wv[wb[i]=G(san[i])]=i;
```

Hàm G là ánh xạ ngược của F , đổi vị trí trong xâu mới về vị trí trong xâu gốc.

Cuối cùng trộn hai danh sách đã sắp:

```
for(i=0,j=0,p=0;i<ta && j<tbc;p++)
sa[p]=c12(wb[j]%3,r,wa[i],wb[j])?wa[i++]:wb[j++];
for(;i<ta;p++) sa[p]=wa[i++];
for(;j<tbc;p++) sa[p]=wb[j++];
```

Hàm $c12$ chính là phép so sánh hai trường hợp đã mô tả ở bước 3.

Nếu gọi độ phức tạp là $f(n)$, các bước sắp xếp và trộn đều tuyến tính, còn lời gọi đệ quy xử lý xâu dài không quá $2n/3$:

$$f(n) = \mathcal{O}(n) + f(2n/3).$$

Suy ra

$$f(n) \leq cn + c(2n/3) + c(4n/9) + \dots \leq 3cn,$$

nên DC3 là thuật toán $\mathcal{O}(n)$.

1.4 So sánh thuật toán nhân đôi và DC3

Độ phức tạp thời gian. Nhân đôi chạy trong $\mathcal{O}(n \log n)$, còn DC3 chạy trong $\mathcal{O}(n)$. Tuy nhiên hằng số của DC3 lớn hơn.

Độ phức tạp bộ nhớ. Cả hai đều dùng $\mathcal{O}(n)$ bộ nhớ. Với cách cài đặt ở trên, nhân đôi cần tổng kích thước mảng khoảng $6n$, còn DC3 khoảng $10n$.

Độ khó cài đặt. Bản nhân đôi ngắn hơn và dễ kiểm soát hơn; bản DC3 dài hơn một chút và có nhiều ánh xạ chỉ số hơn.

Hiệu quả thực tế. Bài gốc thử nghiệm trên NOI-linux, CPU Pentium 4 2.80GHz, không tính thời gian nhập xuất:

N	nhân đôi (ms)	DC3 (ms)
200000	192	140
300000	367	244
500000	750	499
1000000	1693	1248

DC3 có ưu thế thực tế nhất định, còn nhân đôi dễ viết hơn. Khi làm bài, nên chọn theo giới hạn dữ liệu và mức độ rủi ro cài đặt.

2 Ứng dụng của mảng hậu tố

Phần này trình bày các ứng dụng thường gặp. Mẫu chung là xây dựng mảng hậu tố và mảng height, rồi biến điều kiện về xâu con thành điều kiện trên các hậu tố kề nhau hoặc trên các đoạn của mảng height.

2.1 Tiền tố chung dài nhất

Định nghĩa

$$\text{height}[i] = \text{LCP}(\text{suffix}(\text{SA}[i-1]), \text{suffix}(\text{SA}[i])).$$

Với hai vị trí j, k , giả sử $\text{Rank}[j] < \text{Rank}[k]$, ta có tính chất quan trọng:

$$\text{LCP}(\text{suffix}(j), \text{suffix}(k)) = \min_{\text{Rank}[j] < t \leq \text{Rank}[k]} \text{height}[t].$$

Trực giác là khi đi từ hậu tố này tới hậu tố kia trong thứ tự từ điển, mọi cặp kề nhau trên đoạn phải cùng chia sẻ phần tiền tố chung; điểm yếu nhất trên đoạn quyết định độ dài tiền tố chung của hai đầu mút. Hình trong bản gốc minh họa điều này với xâu aabaaaab, khi hỏi LCP của hai hậu tố abaaaaab và aaab.

Tính height trong thời gian tuyến tính. Nếu tính $\text{height}[2], \text{height}[3], \dots, \text{height}[n]$ độc lập bằng cách so ký tự, trường hợp xấu nhất là $\mathcal{O}(n^2)$. Đặt

$$h[i] = \text{height}[\text{Rank}[i]],$$

tức $h[i]$ là LCP của $\text{suffix}(i)$ và hậu tố đứng ngay trước nó trong thứ tự từ điển. Ta có

$$h[i] \geq h[i-1] - 1.$$

Thật vậy, giả sử $\text{suffix}(k)$ đứng ngay trước $\text{suffix}(i-1)$ và LCP của chúng là $h[i-1]$. Nếu $h[i-1] > 1$, sau khi bỏ ký tự đầu, $\text{suffix}(k+1)$ đứng trước $\text{suffix}(i)$ và có LCP ít nhất $h[i-1] - 1$ với $\text{suffix}(i)$. Vì hậu tố đứng ngay trước $\text{suffix}(i)$ là lựa chọn tốt nhất trong các hậu tố đứng trước nó, $h[i]$ không nhỏ hơn giá trị đó. Nếu $h[i-1] \leq 1$ thì bất đẳng thức hiển nhiên.

Ta không cần lưu riêng mảng h ; chỉ cần duyệt i tăng dần và giữ biến k :

```
int rank[maxn], height[maxn];
void calheight(int *r, int *sa, int n)
{
    int i, j, k=0;
    for(i=1; i<=n; i++) rank[sa[i]]=i;
    for(i=0; i<n; height[rank[i++]]=k)
        for(k?k--:0, j=sa[rank[i]-1]; r[i+k]==r[j+k]; k++);
    return;
}
```

Ví dụ 1: truy vấn tiền tố chung dài nhất. Cho một xâu và nhiều truy vấn, mỗi truy vấn hỏi LCP của hai hậu tố. Theo tính chất trên, bài toán trở thành truy vấn giá trị nhỏ nhất trên một đoạn của height. Tiền xử lý RMQ trong $\mathcal{O}(n \log n)$ cho phép trả lời mỗi truy vấn trong $\mathcal{O}(1)$. Nếu dùng tiền xử lý RMQ tuyến tính, toàn bộ tiền xử lý cũng đạt $\mathcal{O}(n)$.

2.2 Các bài toán trên một xâu

Với một xâu đơn, cách làm phổ biến là tính SA và height, sau đó nhìn các xâu con như tiền tố của các hậu tố.

2.2.1 Xâu con lặp

Định nghĩa. Nếu xâu R xuất hiện ít nhất hai lần trong xâu L , ta gọi R là một xâu con lặp của L .

Ví dụ 2: xâu con lặp dài nhất, được phép chồng nhau. Cho một xâu, tìm xâu con lặp dài nhất, hai lần xuất hiện có thể chồng nhau. Độ dài cần tìm chính là giá trị lớn nhất của mảng height. Lý do: LCP của bất kỳ hai hậu tố đều là giá trị nhỏ nhất trên một đoạn của height, nên không thể lớn hơn phần tử lớn nhất của height; ngược lại mỗi $\text{height}[i]$ là LCP của hai hậu tố kề nhau và tương ứng với một xâu lặp. Thời gian sau khi có height là $\mathcal{O}(n)$.

Ví dụ 3: xâu con lặp dài nhất không chồng nhau (PKU 1743). Nhị phân đáp án k . Cần kiểm tra có tồn tại hai xâu con độ dài k bằng nhau và không chồng nhau hay không. Dùng height để chia các hậu tố đã sắp thành từng nhóm, sao cho các hậu tố trong cùng nhóm được nối bởi các giá trị $\text{height} \geq k$. Chỉ các hậu tố cùng nhóm mới có thể có LCP ít nhất k . Với mỗi nhóm, xét giá trị SA nhỏ nhất và lớn nhất; nếu hiệu của chúng ít nhất k , hai lần xuất hiện không chồng nhau. Cách nhóm theo height này rất thường gặp. Tổng thời gian $\mathcal{O}(n \log n)$.

Ví dụ 4: xâu con lặp dài nhất xuất hiện ít nhất k lần (PKU 3261). Bài này tương tự ví dụ 3: nhị phân độ dài, chia nhóm bởi các vị trí có $\text{height} \geq k$, nhưng điều kiện kiểm tra là có nhóm chứa ít nhất k hậu tố. Khi đó tồn tại một xâu con độ dài đang xét xuất hiện ít nhất k lần. Tổng thời gian $\mathcal{O}(n \log n)$.

2.2.2 Số lượng xâu con

Ví dụ 5: số xâu con khác nhau (SPOJ 694, SPOJ 705). Mọi xâu con đều là tiền tố của một hậu tố. Duyệt các hậu tố theo thứ tự

$$\text{suffix}(\text{SA}[1]), \text{suffix}(\text{SA}[2]), \dots, \text{suffix}(\text{SA}[n]).$$

Khi thêm hậu tố $\text{suffix}(\text{SA}[k])$, nó có $n - \text{SA}[k] + 1$ tiền tố. Trong số đó, $\text{height}[k]$ tiền tố đã xuất hiện trong các hậu tố trước, vì chúng là tiền tố chung với hậu tố đứng trước trong thứ tự từ điển. Do đó đóng góp mới là

$$n - \text{SA}[k] + 1 - \text{height}[k].$$

Cộng các giá trị này cho mọi k sẽ được số xâu con khác nhau trong $\mathcal{O}(n)$.

2.2.3 Xâu con đối xứng

Định nghĩa. Một xâu con là đối xứng nếu viết ngược nó vẫn thu được chính nó.

Ví dụ 6: xâu con đối xứng dài nhất (URAL 1297). Liệt kê tâm của xâu đối xứng. Cần xét riêng độ dài lẻ và độ dài chẵn. Cả hai trường hợp đều quy về LCP giữa một hậu tố của xâu gốc và một hậu tố của xâu đảo. Cách làm là ghép xâu gốc, một ký tự phân cách đặc biệt, rồi xâu đảo. Sau đó xây dựng SA, height, và cấu trúc RMQ trên xâu ghép để truy vấn LCP tương ứng với từng tâm. Hình trong bản gốc mô tả việc phản chiếu hai nửa qua tâm trong xâu ghép này. Độ phức tạp là $\mathcal{O}(n \log n)$, hoặc $\mathcal{O}(n)$ nếu RMQ được tiền xử lý tuyến tính.

2.2.4 Xâu con lặp liên tiếp

Định nghĩa. Nếu xâu L thu được bằng cách lặp một xâu S đúng R lần, ta gọi L là xâu lặp liên tiếp, và R là số lần lặp.

Ví dụ 7: chu kỳ của cả xâu (PKU 2406). Cho xâu L , biết nó là S lặp lại R lần; cần tìm R lớn nhất. Liệt kê độ dài chu kỳ k . Trước hết n phải chia hết cho k . Sau đó kiểm tra

$$\text{LCP}(\text{suffix}(1), \text{suffix}(k + 1)) = n - k.$$

Vì một đầu truy vấn luôn là $\text{suffix}(1)$, không cần xây toàn bộ RMQ; chỉ cần biết giá trị nhỏ nhất của height trên các đoạn nối tới $\text{Rank}[1]$. Tổng thời gian $\mathcal{O}(n)$.

Ví dụ 8: xâu con lặp liên tiếp nhiều lần nhất (SPOJ 687, PKU 3693). Liệt kê độ dài khối L . Với mỗi L , tìm số lần nhiều nhất mà một xâu độ dài L có thể xuất hiện liên tiếp. Nếu có ít nhất hai khối liên tiếp, đoạn đó phải phủ một cặp vị trí kề nhau trong dãy

$$0, L, 2L, 3L, \dots$$

Với mỗi cặp Li và $L(i + 1)$, dùng LCP để mở rộng sang phải và dùng LCP trên xâu đảo để mở rộng sang trái. Nếu tổng chiều dài khớp được là K , số lần lặp tại vùng đó là $\lfloor K/L \rfloor + 1$. Với mỗi L , số cặp cần xét là $\mathcal{O}(n/L)$, nên tổng thời gian là

$$\mathcal{O}(n/1 + n/2 + \dots + n/n) = \mathcal{O}(n \log n).$$

2.3 Các bài toán trên hai chuỗi

Với hai chuỗi, cách chuẩn là ghép chúng bằng một ký tự phân cách không xuất hiện trong cả hai, xây dựng SA và height trên chuỗi ghép, rồi chỉ xét các hậu tố thuộc hai chuỗi khác nhau.

2.3.1 Chuỗi con chung

Định nghĩa. Nếu chuỗi L xuất hiện trong cả A và B , ta gọi L là chuỗi con chung của A và B .

Ví dụ 9: chuỗi con chung dài nhất (PKU 2774, URAL 1517). Chuỗi con bất kỳ của một chuỗi là tiền tố của một hậu tố. Vì vậy chuỗi con chung dài nhất của A và B là LCP lớn nhất giữa một hậu tố của A và một hậu tố của B . Sau khi ghép A , ký tự phân cách, rồi B , ta xét các cặp hậu tố kề nhau trong SA. Chỉ khi $SA[i - 1]$ và $SA[i]$ thuộc hai chuỗi gốc khác nhau thì $height[i]$ mới là ứng viên hợp lệ. Giá trị lớn nhất trong các ứng viên này là đáp án. Với $|A|$ và $|B|$ là độ dài hai chuỗi, toàn bộ thuật toán chạy trong $\mathcal{O}(|A| + |B|)$.

2.3.2 Số lượng chuỗi con chung

Ví dụ 10: số chuỗi con chung độ dài ít nhất k (PKU 3415). Cho hai chuỗi A, B , cần đếm số cặp chuỗi con chung có độ dài không nhỏ hơn k , cho phép các chuỗi con bằng nhau được tính nhiều lần. Ví dụ, với $A = xx, B = xx, k = 1$, đáp án là 5. Với $A = aababaa, B = abaabaa, k = 2$, đáp án là 22.

Ý tưởng là cộng, trên mọi cặp hậu tố một từ A và một từ B , phần vượt qua ngưỡng k của LCP. Sau khi ghép hai chuỗi và chia nhóm theo height, cần thống kê nhanh tổng LCP trong mỗi nhóm. Khi quét từ trái sang phải, mỗi lần gặp một hậu tố của B , ta tính nó tạo được bao nhiêu chuỗi con chung độ dài ít nhất k với các hậu tố A đứng trước. Các hậu tố A này được duy trì bằng một ngăn xếp đơn điệu theo giá trị height. Làm thêm một lượt đối xứng cho hậu tố A so với các hậu tố B đứng trước. Chi tiết cài đặt là một bài tập về quét mảng height và ngăn xếp đơn điệu.

2.4 Các bài toán trên nhiều chuỗi

Với nhiều chuỗi, ta ghép tất cả bằng các ký tự phân cách đôi một khác nhau và không xuất hiện trong dữ liệu. Sau đó xây dựng SA, height, rồi dùng height để chia nhóm. Nhiều bài cần nhị phân đáp án.

Ví dụ 11: chuỗi dài nhất xuất hiện trong ít nhất k chuỗi (PKU 3294). Ghép n chuỗi bằng các ký tự phân cách riêng, xây dựng mảng hậu tố. Nhị phân độ dài L ; với mỗi L , chia nhóm bởi các vị trí có $height \geq L$ và kiểm tra có nhóm nào chứa hậu tố đến từ ít nhất k chuỗi gốc hay không. Độ phức tạp $\mathcal{O}(n \log n)$, với n là tổng độ dài.

Ví dụ 12: chuỗi dài nhất xuất hiện ít nhất hai lần không chồng nhau trong mỗi chuỗi (SPOJ 220). Cách làm gần giống ví dụ 11. Với một độ dài đang xét, chia nhóm theo height. Một nhóm hợp lệ nếu trong mỗi chuỗi gốc có ít nhất hai hậu tố thuộc nhóm, và trong từng chuỗi đó, hiệu giữa vị trí bắt đầu lớn nhất và nhỏ nhất không nhỏ hơn độ dài đang xét. Điều kiện hiệu vị trí bảo đảm hai lần xuất hiện không chồng nhau; nếu đề không yêu cầu không chồng nhau thì bỏ kiểm tra này. Độ phức tạp $\mathcal{O}(n \log n)$.

Ví dụ 13: chuỗi dài nhất xuất hiện trực tiếp hoặc sau khi đảo trong mỗi chuỗi (PKU 3294). Điểm khác biệt là mỗi chuỗi có thể xuất hiện ở dạng đảo. Ta chỉ cần thêm bản đảo của từng chuỗi vào chuỗi ghép, vẫn dùng các ký tự phân cách đôi một khác nhau. Sau đó nhị phân đáp án và chia nhóm theo height như

trước. Khi kiểm tra một nhóm, với mỗi xâu gốc chỉ cần có hậu tố thuộc bản thuận hoặc bản đảo. Độ phức tạp vẫn là $\mathcal{O}(n \log n)$.

3 Kết luận

Mảng hậu tố là một cấu trúc dữ liệu rất tốt cho xử lý xâu và là một công cụ mạnh trong nhiều dạng bài. Người học nên nắm vững cách xây dựng mảng hậu tố, mảng hạng và mảng height, đồng thời luyện cách chuyển điều kiện về xâu con thành điều kiện trên các hậu tố liên tiếp hoặc trên các đoạn của height. Các kỹ thuật như chia nhóm theo ngưỡng height, nhị phân đáp án, ghép nhiều xâu bằng ký tự phân cách và dùng RMQ cho LCP xuất hiện lặp lại trong rất nhiều bài toán.

Tài liệu tham khảo

1. Liu Rujia, *Nghệ thuật thuật toán và thi Tin học*, Nhà xuất bản Đại học Thanh Hoa, Bắc Kinh, 2004.
2. Xu Zhilei, luận văn đội tuyển quốc gia IOI 2004, *Mảng hậu tố*.
3. Zhou Yuan, bài tập đội tuyển quốc gia Tin học 2005, *Thuật toán sắp xếp hậu tố tuyến tính*.

Lời cảm ơn

Tác giả cảm ơn CCF đã cung cấp một diễn đàn để trao đổi với mọi người; cảm ơn huấn luyện viên Tang Wenbin của Đại học Thanh Hoa; cảm ơn thầy Zhang Xuedong đã giúp đỡ trong quá trình viết bài; cảm ơn thầy Lin Shenghua của Trường Trung học số 2 Quảng Châu đã hướng dẫn và gợi mở; cảm ơn Fang Zhanpeng của Trường Trung học số 1 Trung Sơn và Jiang Biye của Trường Trung học Kỷ niệm Tôn Trung Sơn đã cung cấp tư liệu về mảng hậu tố.